Software Engineering Lab

Lab 29

# Saga Pattern

Mengelola Transaksi Terdistribusi Tanpa Two-Phase Commit di Microservices




Lukman / lukman-ss

# Ketika Database Terpisah Tidak Bisa Berkomitmen Bersama

Setiap layanan memiliki database sendiri — transaksi ACID klasik runtuh di boundary microservice.

MASALAH DISTRIBUSI TRANSAKSI

### Database Per-Service

Order, Payment, dan Inventory terpisah; tidak ada shared transaction boundary antar layanan.

MASALAH DISTRIBUSI TRANSAKSI

### Two-Phase Commit Rapuh

2PC mengunci semua database secara sinkron hingga seluruh peserta setuju — blocking dan lambat.

MASALAH DISTRIBUSI TRANSAKSI

### Partisi Jaringan Fatal

Satu node down mengunci seluruh transaksi global; availability menurun drastis di bawah network partition.

Di arsitektur microservices modern, setiap service mengelola database sendiri. Saat flow bisnis melibatkan order-creation, payment-processing, dan inventory-reservation secara berurutan, transaksi ACID klasik yang mengandalkan single database transaction langsung runtuh. Dua-phase commit (2PC) muncul sebagai solusi teoretis, namun protokol ini mengunci semua resource secara sinkron hingga seluruh peserta memberikan suara commit — menyebabkan latency tinggi dan vulnerabilitas terhadap network partition.

**When services own their data, distributed transactions need local independence plus compensation.**

# Mengapa Pendekatan Naif Gagal Menangani Transaksi Multi-Layanan

Orkestrasi manual tanpa kompensasi menciptakan inkonsistensi yang tersembunyi dan sulit dideteksi.

```
func (o *OrderService) CreateFullOrder(orderID string) error {
    o.CreateOrder(orderID)     // DB-commit #1
    p.ProcessPayment(orderID)  // DB-commit #2 — jika gagal?
    i.ReserveInventory(...)    // DB-commit #3 — jika gagal?
    o.ApproveOrder(orderID)    // DB-commit #4
}
```

Implementasi naif mencoba memanggil setiap service berurutan tanpa mekanisme rollback lintas domain. Jika payment service gagal setelah order berhasil dibuat, database order mencatat status PENDING namun payment mencatat gagal. State sistem menjadi inkonsisten dan tidak ada jalan mundur otomatis. Pendekatan manual ini juga menyulitkan observabilitas karena tidak ada log eksekusi terpusat yang merekam status setiap langkah.

## KEGAGALAN TANPA SAGA

Payment gagal setelah order dibuat → order tercatat tanpa pembayaran (status PENDING selamanya).
Inventory reserve gagal setelah payment sukses → uang terpotong tapi stok tidak dipesan.
Tidak ada jejak kompensasi; rekonsiliasi manual diperlukan untuk memperbaiki state.

**Manual sequential calls without compensation create silent distributed inconsistency.**

internal/services/services.go • cmd/demo/main.go:11–113

# Mental Model: Estafet Transaksi dengan Kompensasi Mundur

Setiap langkah adalah transaksi lokal independen; jika satu gagal, langkah sebelumnya dikompensasi.

LOCAL TRANSACTION

### 1. Step Execute

Setiap langkah mengeksekusi transaksi lokal ke database-nya sendiri dan berhasil segera.

TRIGGER COMPENSATION

### 2. Step Fail

Jika langkah ke-n gagal, orchestrator memicu compensating transactions pada langkah 1 sampai n-1.

LAST-IN FIRST-OUT

### 3. LIFO Rollback

Kompensasi berjalan dari belakang ke depan: refund payment sebelum cancel order.

ALL OR NOTHING

### 4. Eventual Consistency

Setelah kompensasi selesai, seluruh sistem kembali ke state sebelum saga dimulai.

Bayangkan proses estafet di mana setiap pelari menyelesaikan bagiannya dan langsung mencatat hasil ke database lokal. Jika pelari ketiga terjatuh, pelari satu dan dua berlari mundur untuk mengembalikan posisi awal mereka. Setiap langkah berdiri sendiri sebagai transaksi lokal yang commit-segera; tidak ada locking lintas layanan. Ketika kegagalan terjadi, orchestrator memicu chain of compensations secara Last-In-First-Out untuk memulihkan konsistensi.

**Local transactions commit eagerly; compensations unwind them in reverse order for eventual consistency.**

# Pipeline Orkestrasi: Dari Langkah Eksekusi Hingga Kompensasi

Flow terpusat di Orchestrator: AddStep → Execute → LIFO Compensate pada kegagalan.

## ORCHESTRATOR EXECUTION PIPELINE

```
Client Request
↓
Orchestrator.Execute(ctx)
↓
for each Step in steps:
step.Execute(ctx) → Local DB Write → Commit
on success: append to executed list
on failure: break loop → trigger compensate()
↓
compensate(executed[]):
for i = len-1 down to 0:
executed[i].Compensate(ctx) → Undo DB Write
on comp fail: aggregate errors, log COMPENSATE_FAILED
↓
Return: nil (success) | error (step failed + compensation)
```

Pipeline orkestrasi dimulai ketika client mengirimkan request ke orchestrator. Orchestrator mengiterasi daftar langkah secara sekuensial, mengeksekusi setiap langkah ke database lokal masing-masing service. Pada setiap keberhasilan, langkah dicatat ke dalam stack executed. Jika langkah manapun gagal, loop berhenti dan orchestrator memanggil fungsi compensate() yang berjalan mundur dari stack — memastikan refund terjadi sebelum cancel order. Log Status setiap langkah direkam secara atomik di mutex untuk konsistensi konkuren.

**Forward steps consume; backward compensations undo — orchestrated by a single stateful coordinator.**

internal/saga/orchestrator.go:49–120

# Implementasi Inti: Orchestrator Step & LIFO Compensate

Struktur Step dengan Execute/Compensate pair dan fungsi compensate() berurutan mundur.

```
type Step struct {
    Name       string
    Execute    func(ctx context.Context) error
    Compensate func(ctx context.Context) error
}

func (o *Orchestrator) compensate(ctx context.Context, executed []Step) error {
    for i := len(executed) - 1; i >= 0; i-- {
        step := executed[i]
        if step.Compensate != nil {
            err := step.Compensate(ctx)  // LIFO unwind
            ...
        }
    }
}
```

Struktur data Step mendefinisikan sepasang fungsi Execute dan Compensate untuk setiap transaksi lokal. Orchestrator menyimpan slice Step ini dan mengeksekusinya secara berurutan. Fungsi compensate() melakukan iterasi mundur dari index terakhir ke nol — inilah inti LIFO rollback. Jika compensate() sendiri gagal pada suatu langkah, error di agregasi ke dalam slice compErrors dan dilaporkan bersama error langkah utama.

**Each step owns both its forward action and its undo action — encapsulated as a single Step struct.**

internal/saga/orchestrator.go:19–23,92–112

# Failure Path: ReserveInventory Gagal — Urutan Kompensasi LIFO

Kegagalan stok kosong memicu refund payment sebelum cancel order, sesuai urutan balik stack.

ALUR KEGAGALAN DARI LATAR DEPAN HINGGA BELAKANG

### Langkah 1: CreateOrder

Order dibuat dengan status PENDING. Semantic lock diterapkan pada orderID. Berhasil ✓

ALUR KEGAGALAN DARI LATAR DEPAN HINGGA BELAKANG

### Langkah 2: ProcessPayment

Payment diproses dengan idempotency key. Dana terpotong. Berhasil ✓

ALUR KEGAGALAN DARI LATAR DEPAN HINGGA BELAKANG

### Langkah 3: ReserveInventory GAGAL

Stok item-1 hanya 1 unit, diminta 5. Error: out of stock. Loop berhenti.

ALUR KEGAGALAN DARI LATAR DEPAN HINGGA BELAKANG

### Kompensasi mundur (LIFO)

Compensate Payment (Refund) → Compensate Order (Cancel). State pulih ke awal. ✓

Skenario ini menunjukkan failure path realistis di mana reservasi inventaris gagal karena stok tidak mencukupi. Order sudah dibuat dan payment sudah diproses sebelum kegagalan terdeteksi. Orchestrator kemudian memicu kompensasi LIFO: pertama-tama memproses refund payment, kemudian membatalkan order. Dengan urutan ini, sistem kembali ke keadaan sebelum saga dimulai — payment refund terjadi sebelum order cancel, menjaga konsistensi urutan finansial.

**Compensations must unwind in reverse execution order to prevent orphaned side effects.**

tests/saga_test.go:74–147

# Tabel Perbandingan: Mekanisme Kompensasi vs Gejala Sistem

Memetakan setiap tipe langkah saga ke manifestasi perilaku saat sukses dan gagal.

| Tipe Langkah | Eksekusi Sukses | Eksekusi Gagal | Kompensasi |
|---|---|---|---|
| Compensable Transaction | Local DB commit persisten | Rollback via Compensate() | Refund, Cancel, Release |
| Pivot Transaction | Titik balik tidak bisa dibatalkan | Saga dijamin mencapai final state | Tidak ada kompensasi |
| Retryable Transaction | Post-pivot, dijamin sukses | Idempotent retry | Tidak diperlukan |
| CreateOrder Step | Status PENDING + semantic lock | Rollback: delete lock + CANCELLED | ✓ |
| ProcessPayment Step | Payment recorded + idempotency key | Rollback: delete payment entry | ✓ Refund |
| ReserveInventory Step | Stock decremented | Rollback: stock restored via Release() | ✓ |
| ApproveOrder Step | APPROVED + lock released | kompensasi (final step) | — |

Tabel ini memetakan taksonomi tiga tipe transaksi saga menurut Azure Architecture Center ke implementasi spesifik lab. Compensation step adalah yang paling umum — setiap langkah kecuali ApproveOrder memiliki fungsi Compensate. Pivot transaction merepresentasikan titik balik yang tidak dapat digagalkan, sedangkan retryable transaction menjamin kesuksesan setelah pivot. Dalam lab ini, ApproveOrder berperan sebagai final step tanpa kompensasi karena ia sudah merupakan state terminal.

**Not all saga steps need compensation — final approval marks the pivot of no return.**

# Keputusan Arsitektur: Orkestrasi vs Koreografi

Tiga kriteria teknis menentukan pilihan pola koordinasi saga yang tepat.

PILIHAN POLA KOORDINASI SAGA

### 1. Orkestrasi (Orchestrator)

Koordinator terpusat mengatur urutan, menyimpan log, dan memicu kompensasi. Cocok untuk flow linear dengan dependencies jelas.

PILIHAN POLA KOORDINASI SAGA

### 2. Koreografi (Choreography)

Layanan saling mendengarkan event tanpa koordinator pusat. Cocok untuk flow event-driven dengan loose coupling antar service.

PILIHAN POLA KOORDINASI SAGA

### 3. Hybrid Pattern

Orkestrasi untuk orchestration-critical path; koreografi untuk notification-only paths. Menggabungkan kekuatan kedua pendekatan.

Pemilihan antara orkestrasi dan koreografi bukanlah keputusan hitam-putih. Orkestrasi unggul ketika urutan langkah memiliki dependensi ketat dan perlu logging terpusat, seperti pada flow checkout e-commerce. Koreografi lebih cocok untuk sistem event-driven di mana layanan perlu bereaksi mandiri terhadap event tanpa mengetahui orchestrator. Lab ini mengimplementasikan kedua pola untuk menunjukkan trade-off masing-masing.

**Orchestration gives control; choreography gives autonomy. Choose by dependency density.**

# Bukti Eksekusi: Semua Test Suite Lolos Bersih

Output Go test membuktikan happy path, LIFO rollback, idempotency, semantic lock, dan concurrency safety.

```
=== RUN   TestOrchestrator_HappyPath
    --- PASS: TestOrchestrator_HappyPath (0.00s)
=== RUN   TestOrchestrator_FailureCompensatesLIFO
    --- PASS: TestOrchestrator_FailureCompensatesLIFO (0.00s)
=== RUN   TestPayment_Idempotency
    --- PASS: TestPayment_Idempotency (0.00s)
=== RUN   TestSemanticLock
    --- PASS: TestSemanticLock (0.00s)
=== RUN   TestOrchestrator_Concurrency
    --- PASS: TestOrchestrator_Concurrency (0.00s)
=== RUN   TestChoreography_Flow
    --- PASS: TestChoreography_Flow (0.00s)
=== RUN   TestChoreography_FailureCompensates
    --- PASS: TestChoreography_FailureCompensates (0.00s)
=== RUN   TestOrchestrator_CompensationErrorPropagated
    --- PASS: TestOrchestrator_CompensationErrorPropagated (0.00s)
=== RUN   TestOrchestrator_ContextCancellation
    --- PASS: TestOrchestrator_ContextCancellation (0.00s)
PASS: ok  labs/29-saga-pattern/tests  0.003s
```

Kesembilan test case berjalan mulus tanpa kegagalan satupun. Happy path membuktikan empat langkah sukses mengubah order status menjadi APPROVED dengan stock berkurang 2 unit. Failure test membuktikan LIFO rollback: order kembali CANCELLED, payment di-refund, stock kembali 1. Concurrency test dengan 10 goroutine simultan memastikan atomicity mutex tetap terjaga.

**Clean test suite with zero failures validates both forward execution and backward compensation paths.**

# Batasan Bukti Lab: Yang Dibuktikan vs Belum

Simulasi in-memory tidak membuktikan behavior di bawah network partition atau database crash riil.

YANG TIDAK DIBUKTIKAN

### Network Partition

Tidak ada simulasi network partition antara orchestrator dan service; semua service berjalan dalam proses Go yang sama.

YANG TIDAK DIBUKTIKAN

### Database Crash

Tidak diuji apa yang terjadi jika database crashed di tengah eksekusi step — tidak ada persistence layer riil.

YANG TIDAK DIBUKTIKAN

### Persistensi Log

Step log hanya disimpan di memori; tidak ada audit trail yang persisten ke disk atau database.

YANG TIDAK DIBUKTIKAN

### Distributed EventBus

EventBus di choreography berjalan in-memory; tidak diuji dengan message broker riil seperti Kafka atau RabbitMQ.

Lab ini membuktikan kebenaran logika komputasional dan concurrency safety dalam lingkungan in-memory. Namun, batasan tegas harus diakui: tidak ada simulasi network partition, database crash, atau persistence layer riil. Dalam produksi, kegagalan kompensasi bisa terjadi karena service unavailable — scenario yang tidak tercakup oleh test suite saat ini. Dead-letter queue dan manual reconciliation adalah countermeasures yang diperlukan di luar cakupan lab.

**In-memory proofs validate logic; production requires persistence, observability, and DLQ handling.**

# Trade-off Arsitektur: Availability vs Isolation

Saga mengorbankan isolasi ACID demi ketersediaan tinggi — eventual consistency adalah harga yang harus dibayar.

| Dimensi | ACID (Monolitik) | Saga Pattern | Dampak Produksi |
|---|---|---|---|
| Isolasi (I dalam ACID) | serializable penuh | tidak ada isolasi lintas service | dirty reads, lost updates mungkin terjadi |
| Ketersediaan | blocking 2PC | non-blocking, each step commit segera | saga jauh lebih available di bawah load tinggi |
| Latensi | satu round-trip ke semua peserta | N round-trips sekuensial + kompensasi | total latency lebih tinggi per transaksi |
| Konsistensi | strong consistency | eventual consistency | window inconsistency antar step execute sampai kompensasi |
| Complexity | database engine handles it | application-level orchestration | developer wrote compensation logic manually |
| Observability | transaction logs | step-level logs + compensation | lebih transparan, lebih mudah debug |

Saga menukar isolation level tinggi menjadi availability tinggi. Setiap langkah committed segera tanpa lock lintas service, sehingga tidak ada blocking antar layanan. Namun, window inconsistency muncul antara step execute dan kompensasi — order bisa tercatat APPROVED sementara payment belum diproses. Developer harus menulis compensation logic secara eksplisit, menambah complexity aplikasi dibanding solusi monolitik.

**Availability over isolation: commit early, compensate later, accept eventual consistency.**

# Pipeline Konkurensi: Goroutine → Validate → Mutex → Append Error

Orchestrator menggunakan sync.Mutex untuk melindungi step list dan log agar aman dari concurrent Execute.

```
func (o *Orchestrator) AddStep(step Step) {
    o.mu.Lock()
    defer o.mu.Unlock()
    o.steps = append(o.steps, step)
}

func (o *Orchestrator) Execute(ctx context.Context) error {
    o.mu.Lock()
    steps := make([]Step, len(o.steps))
    copy(steps, o.steps)  // snapshot tanpa lock holder
    o.mu.Unlock()
    // ...execute steps without holding mu during step calls
    o.mu.Lock()
    o.logs = append(o.logs, StepLog{Name: step.Name, Status: ...})
    o.mu.Unlock()
}
```

Keamanan konkuren dicapai melalui strategi snapshot-and-release. Saat Execute dipanggil, orchestrator menyalin slice steps ke variabel lokal sementara mutex dilepaskan, menghindari blocking selama step execution berlangsung. Log append dan read dilindungi mutex individual untuk memastikan konsistensi pembacaan. Test concurrency menjalankan 10 goroutine simultan dengan mutex dan race detector Go melaporkan status bersih tanpa data race.

## KONTRAK KEAMANAN

Mu memegang slice steps saat AddStep danExecute — mencegah race pada read.
Snapshot steps saat Execute: salinan sementara dilepaskan sebelum step call.

**Snapshot-then-release mutex pattern prevents deadlock while ensuring log consistency under concurrency.**

Hasil: go test -race laporan clean pada TestOrchestrator_Concurrency (10 workers).

# Before vs After: Dari Manual Calls ke Saga Orkestrasi

Perbandingan visual antara implementasi naif tanpa kompensasi dan implementasi saga dengan LIFO rollback.

PERBANDINGAN SEBELUM DAN SESUDAH SAGA

## BEFORE: Manual Calls

Orde → Payment → Inventory → Approve. Gagal di Inventory: order PENDING, payment tercatat, stok untouched. Tidak ada recovery otomatis. State inconsistency permanen.

PERBANDINGAN SEBELUM DAN SESUDAH SAGA

## AFTER: Saga Orchestration

Orde → Payment → Inventory(GAGAL) → Compensate Payment(refund) → Compensate Order(cancel). State pulih sempurna. Log lengkap记录了每一步状态。

PERBANDINGAN SEBELUM DAN SESUDAH SAGA

## BEFORE: No Observability

Tidak ada log terpusat; harus trace masing-masing service log untuk reconstruct failure.

PERBANDINGAN SEBELUM DAN SESUDAH SAGA

## AFTER: Centralized Step Log

Orchestrator mencatat STATUS_EXECUTED, STATUS_FAILED, STATUS_COMPENSATED untuk setiap langkah. Debugging satu command: orch.Logs().

Perbandingan sebelum dan sesudah menunjukkan transformasi fundamental dari state management manual ke orchestration terstruktur. Sebelum saga, kegagalan di satu service meninggalkan state inconsistency yang memerlukan intervensi manual. Setelah saga, orchestrator secara otomatis mencatut log status dan memicu kompensasi berurutan. Setiap langkah memiliki jejak audit trail yang memungkinkan debugging cepat tanpa perlu menggabungkan log dari multiple service.

**Orchestration replaces chaos with traceable state transitions and automatic recovery.**

# Service Layer: Semantic Lock, Idempotency, dan Atomic Stock Mutation

Setiap service memiliki mekanisme proteksi mandiri sebelum saga menyentuhnya.

### SEMANTIC LOCK (ORDERSERVICE)

func CreateOrder checks s.locks[orderID] — menolak duplikasi saat saga masih berjalan.
ApproveOrder menghapus lock via delete(s.locks, orderID) setelah approved.
CancelOrder juga menghapus lock agar order yang dikompensasi bisa dibuat ulang.

### IDEMPOTENCY KEY (PAYMENTSERVICE)

processedID map tracks已processed payment IDs.
func ProcessPayment: if processedID[id] == true → return nil (idempotent success).
RefundPayment: delete(s.payments, id) — aman dipanggil berkali-kali.

### ATOMIC STOCK MUTATION (INVENTORYSERVICE)

Reserve: atomic stock decrement under mutex — stock[item] -= qty, reserved[item] += qty.
Release: atomic stock restore under mutex — reserved[item] -= qty, stock[item] += qty.
Kedua operasi dilindungi sync.Mutex tunggal — nol race condition pada concurrent access.

Service layer bukan sekadar storage abstraction — setiap service mengandung mekanisme proteksi mandiri. OrderService menggunakan semantic lock untuk mencegah double-creation order dalam satu saga instance. PaymentService mengandalkan idempotency key untuk menangani retry berulang pada payment step. InventoryService menjamin atomicity stock mutation melalui mutex tunggal. Tanpa proteksi ini, saga kompensasi sendiri bisa menimbulkan race condition.

**Service-level guards complement saga-level compensation — defense in depth across layers.**

# Choreography Pattern: Event Bus Decoupled via Pub/Sub

Setiap layanan subscribe ke event tertentu dan publish event setelah proses selesai.

```
type EventBus struct {
    mu       sync.RWMutex
    handlers map[EventType][]EventHandler
}

func (b *EventBus) Subscribe(t EventType, h EventHandler) {
    b.mu.Lock()
    b.handlers[t] = append(b.handlers[t], h)
    b.mu.Unlock()
}

func (b *EventBus) Publish(ctx context.Context, e Event) {
    b.mu.RLock()
    handlers := append([]EventHandler(nil), b.handlers[e.Type]...)
    b.mu.RUnlock()
    for _, h := range handlers { h(ctx, e) }
}
```

Choreography pattern menghilangkan koordinator terpusat sepenuhnya. Setiap layanan mendengarkan event dari layanan lain dan bereaksi sesuai handler yang terdaftar. Event bus menggunakan RWMutex untuk keamanan konkuren: Subscribe memegang write lock, Publish memegang read lock sementara menyalin handler slice. Pola ini menghasilkan loose coupling tetapi mengurangi observabilitas karena tidak ada central log eksekusi.

### FLOW CHOREOGRAPHY

OrderCreated event → PaymentService.ProcessPayment →
PaymentCompleted/PaymentFailed
PaymentCompleted → InventoryService.Reserve →
InventoryFailed/PaymentFailed → PaymentService.RefundPayment +
OrderService.CancelOrder

**Decentralized event-driven coordination trades observability for service autonomy.**

# Bukti Invarian: Idempotency Payment dan Semantic Lock Order

Dua test case mengonfirmasi bahwa state mutation tetap konsisten di bawah kondisi edge case.

```
func TestPayment_Idempotency(t *testing.T) {
    paymentSvc := services.NewPaymentService()
    id := "tx-1"
    // First call: records payment
    paymentSvc.ProcessPayment(id, 100, false)
    // Second call with same ID: idempotent no-op
    err := paymentSvc.ProcessPayment(id, 100, false)
    // Must succeed (nil error) — double charge prevented
    if err != nil { t.Fatalf(...) }
}

func TestSemanticLock(t *testing.T) {
    orderSvc := services.NewOrderService()
    id := "ord-lock"
    orderSvc.CreateOrder(id)  // sets locks[id] = true
    err := orderSvc.CreateOrder(id)  // should FAIL
    if err == nil { t.Fatal("second create should fail due to semantic lock") }
}
```

Test idempotency membuktikan bahwa payment service menerima panggilan berulang dengan ID yang sama tanpa melakukan double charge. Test semantic lock membuktikan bahwa order service menolak kreasi order duplikat selama saga instance masih berjalan — mencegah race condition antara concurrent saga instances yang menggunakan orderID sama.

## INARIAN YANG DIBUKTIKAN

Idempotency: ProcessPayment dengan ID sama tidak menduplikasi charge.
Semantic lock: CreateOrder dengan orderID yang sama ditolak selama lock masih aktif.

**Idempotency and semantic locks are the cornerstones of reliable saga step execution.**

tests/saga_test.go:149–171

# Bukti LIFO Log: Verifikasi Urutan Status Step pada Kompensasi

Test menegaskan log records tepat 5 entri: EXECUTED, EXECUTED, FAILED, COMPENSATED, COMPENSATED.

```
func TestOrchestrator_FailureCompensatesLIFO(t *testing.T) {
    // ...setup: CreateOrder, ProcessPayment, ReserveInventory(fail)...
    err := orch.Execute(ctx)
    // Verify state recovery
    if orderSvc.GetOrder(orderID) != services.OrderCancelled {
        t.Fatalf("expected order cancelled, got %v", orderSvc.GetOrder(orderID))
    }
    if paymentSvc.HasPayment(orderID) {
        t.Fatal("expected payment refunded")
    }
    // Verify log order (5 entries exactly)
    logs := orch.Logs()
    expectedOrder := []saga.StepStatus{
        saga.StatusExecuted, saga.StatusExecuted, saga.StatusFailed,
        saga.StatusCompensated, saga.StatusCompensated,
    }
    for i, l := range logs {
        if l.Status != expectedOrder[i] { ... }
    }
}
```

Test ini memverifikasi tiga hal sekaligus: state recovery (order CANCELLED, payment refunded), stock integrity (stock tetap 1), dan urutan log yang tepat. Enam entri log — dua EXECUTED, satu FAILED, dua COMPENSATED — membentuk sequence yang membuktikan LIFO rollback bekerja sesuai desain. Urutan ini penting untuk debugging: engineer bisa merekonstruksi kronologi exact failure.

## LOG EKSPEKTASI

Idx 0: CreateOrder → EXECUTED (order PENDING)

**Log sequence integrity is as important as state recovery — every compensation must be traced.**

Idx 3: ProcessPayment → COMPENSATED (refund executed)

test/saga_test.go:74–147

# Bukti Context Cancellation: Kompensasi Saat Saga Dibatalkan

Test memastikan step yang sudah dieksekusi dikompensasi saat context dibatalkan di tengah jalan.

```
func TestOrchestrator_ContextCancellation(t *testing.T) {
    ctx, cancel := context.WithCancel(context.Background())
    orch := saga.NewOrchestrator()
    step1Compensated := false
    orch.AddStep(saga.Step{
        Name: "Step1",
        Execute: func(ctx context.Context) error {
            cancel() // cancel context before step 2
            return nil
        },
        Compensate: func(ctx context.Context) error {
            step1Compensated = true
            return nil
        },
    })
    orch.AddStep(saga.Step{
        Name: "Step2",
        Execute: func(ctx context.Context) error {
            t.Fatal("step 2 should not execute when context cancelled")
            return nil
        },
    })
    err := orch.Execute(ctx)
    if !step1Compensated { t.Fatal("expected step 1 compensated") }
}
```

Test ini memvalidasi perilaku saga ketika context dibatalkan di tengah eksekusi. Context cancellation dideteksi pada select case di loop Execute, menghentikan iterasi sebelum step berikutnya dijalankan. Meskipun context dibatalkan, compensate() tetap dipanggil dengan context.Background() — memastikan kompensasi tidak terhenti oleh cancellation propagation. Step yang sudah dieksekusi pasti dikompensasi, mempertahankan invarian atomicity.

**Context cancellation must not abandon in-flight compensations — use detached context for rollback.**

INARIAN CONTEXT CANCELLATION

tests/saga_test.go:353–385 • internal/saga/orchestrator.go:57–69

# Checklist Implementasi Saga Pattern yang Benar

Delapan poin wajib sebelum saga pattern siap deployment ke production.

## CHECKLIST IMPLEMENTASI

[x] Setiap step memiliki fungsi Compensate yang valid (tidak nil untuk non-final steps).
[x] Setiap step Execute dan Compensate adalah idempotent — aman dipanggil berulang.
[x] Semantic lock diterapkan pada entity yang bisa diakses concurrent saga instances.
[x] Orchestrator menggunakan snapshot pattern untuk concurrent-safe step execution.
[x] Context cancellation handled: compensate() dipanggil dengan context independent.
[x] Step log mencatat semua status: PENDING, EXECUTED, FAILED, COMPENSATED, COMPENSATE_FAILED.
[x] go test -race dijalankan dan melaporkan clean — zero data race.
[x] Dead-letter queue planned untuk compensation failures yang tidak bisa otomatis recover.

Checklist ini merangkum delapan persyaratan kritis untuk implementasi saga pattern yang production-ready. Poin paling sering terlewat adalah idempotensi kompensasi — jika compensate() dipanggil dua kali, sistem harus tetap stabil. Poin keenam tentang dead-letter queue juga krusial: compensation failure permanen memerlukan mekanisme manual intervention yang harus direncanakan sebelum deployment.

**Idempotent compensations and observable logs are non-negotiable for production sagas.**

internal/saga/orchestrator.go • tests/saga_test.go • content/02-master-draft.md:143–148

# Anti-Pattern Saga: Yang Harus Dihindari di Production

Empat kesalahan desain umum yang menyebabkan data corruption atau infinite rollback loops.

ANTI-PATTERNS YANG HARUS DIHINDARI

### Non-Idempotent Compensate

Compensate() memanggil external API tanpa idempotency key → refund ganda saat retry kompensasi.

ANTI-PATTERNS YANG HARUS DIHINDARI

### Circular Compensation Dependency

Step A compensate memanggil Step B execute → infinite loop kompensasi timbal balik.

ANTI-PATTERNS YANG HARUS DIHINDARI

### Missing Compensate for Final Step

Final step (ApproveOrder) tidak punya Compensate, tapi jika di tengah jalan di-rollback, approval permanen.

ANTI-PATTERNS YANG HARUS DIHINDARI

### Blocking Long-Running Compensate

Compensate() memanggil synchronous HTTP ke external service → orch stall, lock held, deadlock.

Empat anti-pattern ini paling umum ditemukan di implementasi saga production. Non-idempotent compensate adalah penyebab utama double-refund yang merugikan financial reconciliation. Circular dependency antara kompensasi step menciptakan infinite loop yang melumpuhkan orchestrator. Missing compensate pada final step berarti approval state permanen meskipun step lain dikompensasi. Blocking long-running compensate mengunci goroutine orchestrator dan mencegah step selanjutnya dieksekusi.

**Every compensation must be idempotent, non-blocking, and free from circular dependencies.**

# Takeaway Praktis: Command & Runbook untuk Saga Pattern

Kumpulan command reproduksi dan prinsip operasional untuk saga di lingkungan production.

```
# Jalankan seluruh test suite
go test -v ./...

# Verifikasi race condition
go test -race -v ./...

# Demo interaktif (happy path + rollback)
go run ./cmd/demo

# Production runbook:
# 1. Monitor orch.Logs() untuk STATUS_FAILED / STATUS_COMPENSATE_FAILED
# 2. Alert on COMPENSATE_FAILED → trigger DLQ processing
# 3. Manual reconciliation via admin console for stuck compensations
# 4. Regular healthcheck: verify all in-flight sagas reached terminal state
```

Takeaway praktis untuk engineer yang mengadopsi saga pattern di production: command di atas adalah titik awal reproduksi mandiri. Prinsip operasional ketiga poin terakhir merupakan panduan monitoring yang harus diimplementasikan sebelum go-live. Compensation failure yang masuk ke DLQ harus diproses secara manual oleh on-call engineer dengan konsol admin — tidak ada fully-automatic resolution untuk semua kasus.

## PRINSIP OPERASIONAL

Kompensasi adalah transaksi biasa — perlakukan dengan sama seriusnya seperti eksekusi forward.
Setiap saga instance memiliki SagaID unik untuk tracing end-to-end.
Dead letter queue diperlukan untuk compensation failure yang tidak bisa auto-retry.

**Runbooks matter as much as code — compensation failure handling requires operational discipline.**

# Prinsip Arsitektural Jangka Panjang: Eventual Consistency sebagai Kontrak

Saga pattern mengajarkan bahwa konsistensi eventual adalah harga yang sah untuk availability tinggi.

EACH SERVICE OWNS DATA

### 1. Local Autonomy

Setiap service mengelola database sendiri; tidak ada shared transaction boundary.

COMMIT BEFORE NEXT STEP

### 2. Eager Local Commit

Setiap langkah committed segera ke database lokal — tidak ada pending write antar step.

UNDO AS FIRST-CLASS OPERATION

### 3. Explicit Compensation

Kompensasi adalah operation explicit, bukan implicit rollback — didesain, di-test, di-monitor.

LOGS ADALAH SOURCE OF TRUTH

### 4. Observable State

Step log adalah satu-satunya sumber kebenaran untuk debugging distributed failure.

ACCEPT PARTIAL FAILURE

### 5. Resilient Design

Sistem diterima sebagai entitas yang bisa gagal parsial — compensations dirancang untuk recover.

Prinsip arsitektur ini berlaku jauh melampaui implementasi lab. Sagas bukan sekadar pola teknis — mereka mewakili filosofi desain sistem terdistribusi yang menerima eventual consistency sebagai kontrak fundamental. Setiap prinsip di atas mengarahkan keputusan arsitektural: dari pemilihan database per service hingga desain monitoring dan alerting system. Engineer yang memahami ini akan menghindari godaan untuk kembali ke ACID monolitik.

**Eventual consistency is not a compromise — it is the architecture of scale.**

# Eksperimen kecil, interpretasi yang teliti.

## Distributed consistency is earned through deliberate design, not inherited from databases.

### Software Engineering Lab 29 - Saga Pattern

Repo: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/29-saga-pattern
Commit SHA: ba4b6216
internal/saga/orchestrator.go — Orchestrator dengan LIFO compensation
internal/saga/choreography.go — Event bus untuk pattern choreography
internal/services/services.go — Order, Payment, Inventory dengan semantic lock & idempotency
tests/saga_test.go — 9 test cases: happy path, LIFO rollback, idempotency, concurrency
cmd/demo/main.go — Interactive CLI demonstration

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/29-saga-pattern`

### Lukman / lukman-ss

Repo: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/29-saga-pattern
Commit SHA: ba4b6216
internal/saga/orchestrator.go — Orchestrator dengan LIFO compensation
internal/saga/choreography.go — Event bus untuk pattern choreography
internal/services/services.go — Order, Payment, Inventory dengan semantic lock & idempotency
tests/saga_test.go — 9 test cases: happy path, LIFO rollback, idempotency, concurrency
cmd/demo/main.go — Interactive CLI demonstration
Semua test suite: PASS (Race-Clean, Go 1.22+)
Invarian terbukti: LIFO rollback, idempotency, semantic lock, concurrency safety.
Batasan: in-memory simulation — belum mencakup network partition atau persistence crash.